西 安 邮 电 大 学

AI 全栈开发实验

题目：算法优化实验

组　　号：**23**

组员信息：

学号	姓名	分工
2610321101	何朝杨	实验三报告整合、排版与图表引用
2610321103	杜军博	推理加速评测复算与交付（速度精度验收、132 查询复算）
2610321135	王陆洲	模型导出与推理测量（ONNX、ORT/TRT 测速、FLOPs）
2610320046	张梓萱	训练数据与划分核验、成果核验与交付
2610320033	刘凡思雨	结果数据核对与模型/索引版本记录
2610321168	秦鑫洋	交付材料整理与报告校对

时　　间：**2026.10.25**

一、实验目的

本实验承接实验一的电商商品图文数据与实验二微调得到的 Chinese-CLIP 中文图文检索模型，对实验二的双塔模型做算法优化与部署验证，具体目的如下：

1. 掌握模型轻量化核心技术的实现与边界：以实验二微调权重为对象，理解动态量化[1,2]、结构化剪枝与知识蒸馏[3]的实现方式与适用条件，并通过 INT8 动态量化的完整实测认清其在本任务上的精度代价；
2. 掌握 ONNX 模型转换与跨框架计算图优化：将双塔分别导出 ONNX[4]（图像塔固定 224×224、文本塔序列长度动态），理解算子融合、图优化等级与动态批处理的作用；
3. 掌握推理加速方法：在同硬件配对口径下比较 PyTorch 与 ONNX Runtime[5]（CPU/CUDA 执行提供器）、TensorRT[6]后端的延迟与吞吐，定位真正有效的加速路径；
4. 掌握精度-速度-资源的量化评估方法：以 934 条双塔文搜图 Recall@K 差衡量精度一致性，以参数量、理论 FLOPs 与文件体积衡量资源，形成“性能-精度-资源”证据链；
5. 理解异构硬件适配与工程约束：处理 CUDA、ONNX Runtime 与 TensorRT 的版本耦合，FP16 图层面对齐[7]，以及 CPU 多线程对齐等部署问题；
6. 形成可复现的评测与交付规范：把测量凭证、复算脚本、中文图表与限制说明分离归档，使结论可由脚本从已核验输入重新计算，供报告与答辩复核。

二、实验环境

软硬件环境

1. 硬件：GPU 训练机一台，CPU 后端配对测速在 8 核上进行且要求 PyTorch 与 ONNX Runtime 均为 8 线程；为满足“同硬件配对”原则，速度对比一律在同设备、同精度内进行，不做 CPU 与 GPU 之间的绝对耗时换算。源测速凭证未单独记录 GPU 型号，本轮不补写（见五（六））；
2. 操作系统/容器：模型推理在训练机的 Linux 容器内完成，复算与绘图在 Windows 11 主机（Python 3.12.13）上进行；容器内 Python 3.11；
3. 软件与框架：训练机 torch 2.7.1+cu128、onnx 1.23.2、onnxruntime-gpu 1.22.0、TensorRT 10.x（cu12）、CUDA 12.8；复算与绘图端只需 numpy、matplotlib、Pillow，不要求 torch/CUDA/TensorRT；
4. 模型：实验二微调的 Chinese-CLIP ViT-B-16（图像塔）[8] + RoBERTa-wwm-ext-base-chinese（文本塔）[9,10]，checkpoint 为 `epoch3.pt`（含优化器动量约 2100.85 MB）；
5. 数据：实验一产出的 9 类 934 条 test 图文对（文搜图、候选池同为 934），与实验二的评测口径完全一致；另保留旧索引的 132 条 AI 试点查询用于历史端到端参考。

三、实验内容与步骤

任务书阶段 1 列出六个可选优化维度（量化、剪枝、蒸馏、推理加速、资源占用、鲁棒性），要求至少完成一个；阶段 2（选做）为优化与部署验证（计算图优化、硬件加速与跨平台部署）。本实验选择 **3.1.4** 推理加速 作为主线并顺带交付 **3.2.1** 计算图优化，其余维度或已冻结、或本轮未纳入，说明如下。

3.1 模型优化

本实验选择的优化维度是“性能”维度下的推理加速（3.1.4），叠加以 ONNX 为载体的计算图优化（3.2.1）。选择原因：

1. 与实验二落点直接衔接：实验二已把微调权重重新编码并接入 FAISS 检索索引[11]，检索服务的主要算力开销集中在双塔编码，推理加速的收益可直接作用于既有链路；
2. 不改结构、不重训、风险可控：推理加速只替换执行后端与计算图，不改变模型结构与参数，因此参数量与理论 FLOPs 不变，可在保持 R@1 的前提下压低延迟；
3. 指标可判定：任务书为该维度给出明确硬指标（延迟降低 ≥40%、吞吐提升 ≥50%、精度差 ≤1pp），阶段 2 的 3.2.1 又给出加速比 ≥1.2、精度差 ≤0.5pp 的口径，二者可用同一批数据一并交付；
4. 量化维度已先行实测且代价过高：六种动态 INT8 配置中最好的一档 R@1 仍下跌 13.06 个百分点，远未达到任务书“精度损失 ≤2%”的要求，故量化仅作为历史探索保留、不再作为主线；
5. 剪枝、蒸馏、资源占用与鲁棒性本轮未纳入：受学时与训练机可用时间限制，本轮未执行这三类任务，正文如实标注，不作对应结论。

3.2 优化与部署验证

推理优化与高效部署的思路分四步：

1. 模型格式转换：双塔分别导出 ONNX。图像塔固定 224×224、仅 batch 动态，便于 TensorRT 做层融合；文本塔 batch 与序列长度均动态，以适配实验一属性模板文本的长度变化（评测时按上下文长度 52 定长）；
2. 精度格式转换：先导出 FP32 ONNX，再在图层面转 FP16（保留输入输出为 FP16），避免 cn_clip 中把 LayerNorm 写死为 FP32 导致的混合精度报错；
3. 推理引擎配置：以 ONNX Runtime 的 `ORT_ENABLE_ALL` 图优化为统一入口，分别使用 CPUExecutionProvider（FP32）、CUDAExecutionProvider（FP16）与 TensorrtExecutionProvider（FP16，层融合 + 引擎缓存），动态批处理覆盖 1–32；
4. 部署验证：同硬件配对测速（batch 1 与 batch 32），并用与实验二相同的 934 条双塔文搜图比较 R@K 与同源 FP32 基线的绝对差。

以下按需求分析、设计思路、代码实现、测试优化四个方面展开。

3.3 需求分析

检索推荐与上架 Agent 在部署阶段对模型侧提出的需求可归纳为四点：

1. 部署性能需求：检索请求的算力开销集中在双塔编码，尤其图像塔（理论 FLOPs 33.70 G，占双塔 42.63 G 的约 79%），需要在保持精度前提下降低其 forward 延迟（单样本与 batch 32 两种规模）；
2. 精度保持需求：加速不得显著损失双塔文搜图精度，要求与同源同硬件的 FP32 基线相比 R@1 绝对差 ≤1 个百分点（百分点 pp，非相对百分比）；
3. 跨后端可移植需求：同一份 ONNX 图应可在 CPU 与 GPU 上以不同执行提供器运行，并支持动态批处理 1–32，便于按硬件条件选择部署路径；
4. 工程可复现需求：测量必须同硬件配对、线程对齐，并保留可追溯到确定输入、版本与哈希的凭证，使结论能被脚本重新计算。

此外有一个硬约束：本轮交付沿用训练机已核验的测量凭证完成复算与绘图，没有在本机新增模型推理，也没有重训或改变模型结构。

3.4 设计思路

整体推理加速与部署流水线如图 1 所示：实验二的微调权重经双塔 ONNX 导出与图层面 FP16 转换后，分别接入 ORT-CPU、ORT-CUDA 与 TensorRT 三种后端，在同硬件配对口径下测速，并在 934 条双塔文搜图上比对 R@K，最后生成表、图与复算入口并归档。



图 1 推理加速与双塔 ONNX 部署流水线

主要设计决策如下：

1. 双塔分别导出而非整体单图：图像塔输入尺寸固定、便于层融合与静态 shape 优化，文本塔需保留动态序列长度；分塔导出也便于定位精度差异来自哪一塔；
2. 先导 FP32 再图层面转 FP16：cn_clip 的 LayerNorm 在模型定义中强制 FP32，直接对半精度权重导出会触发 mixed dtype 错误，改在 ONNX 图层面转换可绕开且不损失 IO 类型控制；
3. 后端三级对照、由便宜到贵：ORT-CPU（零安装零风险）→ ORT-CUDA → TensorRT，任一步达标即停止，避免为追逐更强后端付出不必要的环境成本；
4. 精度一致性使用同源基线：以自研评测的 PyTorch-FP32 结果（R@1 = 58.5653）为共同基线，不混用实验二官方脚本的 58.67（官方 58.67 与自研 58.5653 相差约 0.1pp，来自特征传递与 top-k 实现等口径差异）；
5. 测量与报告解耦：模型侧脚本产出 JSON 凭证并保留原件，报告侧脚本仅从凭证复算数字并绘图，避免“手工改数”造成文档与 JSON 不一致。

3.5 代码实现

3.5.1 模型导出与精度转换

`scripts/optimize/export_onnx.py` 负责单塔导出并校验 PyTorch 与 ONNX Runtime 输出的一致性。图像塔以固定 224×224 输入、仅 batch 动态；文本塔包装为只吃 `input_ids` 与 `attention_mask` 的模块，batch 与序列长度均动态（见代码 1），opset 均为 17。导出后用固定随机种子比较两路嵌入的余弦相似度，要求 ≥0.999，图像塔实测余弦 1.000000、最大绝对差 4e-6。

```
                torch.onnx.export(
                    wrapper, (dummy_ids, dummy_mask), str(out_path),
                    input_names=["input_ids", "attention_mask"],
                    output_names=["embedding"],
                    dynamic_axes={
                        "input_ids": {0: "batch", 1: "seq"},
                        "attention_mask": {0: "batch", 1: "seq"},
                        "embedding": {0: "batch"},
                    },
                    opset_version=17, do_constant_folding=True,
                )
```

代码 1　文本塔 ONNX 导出：batch 与序列长度均动态

FP16 不在 PyTorch 侧直接生成，而在 ONNX 图层面转换（见代码 2）：`keep_io_types=False` 使输入输出也变为 FP16，因此推理时必须喂 FP16 数据，否则报 `Unexpected input data type`。

```
        model = onnx.load(str(fp32_path))
        model_fp16 = convert_float_to_float16(model, keep_io_types=False)  #
        onnxruntime.transformers.float16
        onnx.save(model_fp16, str(fp16_path))
```

代码 2 FP32 ONNX 图层面转 FP16

3.5.2 推理后端与计算图配置

`scripts/optimize/bench_runtime.py` 统一构建 ONNX Runtime 会话：图优化等级取 `ORT_ENABLE_ALL`，CPU 侧显式设定 `intra_op_num_threads`，TensorRT 侧显式给出 FP16、引擎缓存与工作空间配置（见代码 3），否则每次运行都会重新构建引擎（首次约 5–20 分钟）。

```
providers = [(provider, {
    "trt_fp16_enable": "True" if precision == "fp16" else "False",
    "trt_engine_cache_enable": "True",
    "trt_engine_cache_path": str(cache_dir),
    "trt_timing_cache_enable": "True",
    "trt_max_workspace_size": str(4 * 1024 ** 3),
})]
sess = ort.InferenceSession(str(onnx_path), sess_options=opts, providers=providers)
```

代码 3 TensorRT 执行提供器配置（FP16 + 引擎缓存）

会话构建后检查请求的 provider 是否在 `sess.get_providers()` 中，若发生回退则标记该次结果无效，避免把回退到 CPU 的结果记成 TensorRT 实测。

3.5.3 测速与精度一致性评测

测速只覆盖图像塔 forward（不含预处理/后处理），预热 10 次后计时 20 次取中位数（见代码 4）；吞吐按 批大小 / 中位延迟 计算。CUDA 侧在计时前后调用 `torch.cuda.synchronize()`，避免异步执行造成低估。P90 作为同轮尾延迟写入表格，但不据此绘制置信区间。

```
                for _ in range(iters):
                    if device == "cuda":
                        torch.cuda.synchronize()
                    t0 = time.perf_counter()
                    visual(x)
                    if device == "cuda":
                        torch.cuda.synchronize()
                    times.append(time.perf_counter() - t0)
                median_ms = statistics.median(times)
                p90_ms = sorted(times)[int(len(times) * 0.9) - 1]
```

代码 4 测速循环与统计（中位数 / P90）

精度一致性由 `scripts/optimize/eval_onnx.py` 完成：两塔均走 ONNX，在 934 条 test 文本与 934 张候选图片上做文搜图 top-10（第 i 段文本的标准答案为第 i 张图），特征 L2 归一化后按 `text @ image.T` 排序，命中对角线即为命中，取 R@1/R@5/R@10 与 `mean_recall`，并与同源 FP32 基线求绝对差。`scripts/optimize/bench_flops.py` 用 `FlopCounterMode` 只统计矩阵乘/卷积类算子，得到两塔参数量与理论 FLOPs。

3.5.4 复算与绘图流水线

为让报告数字可独立复核，仓库另有一套“证据回放”链路：`evaluation/retrieval/code/evaluate_acceleration.py` 读取训练机凭证（测速、精度、规模、导出、文件体积、历史查询排序），按配置复算加速比、延迟降低、吞吐提升与 R@K 差值，并施加阈值判定；`render_acceleration_charts.py` 从复算表格独立重绘中文图；`evaluation/retrieval/tests/test_acceleration.py` 用 8 项逻辑测试覆盖配对比值、provider 回退拒绝、CPU/GPU 跨设备拒绝、线程不对齐拒绝、基线与候选池一致、阈值含等号、零延迟拒绝与相对路径限制。复现入口有两种：

```
python evaluation/retrieval/code/evaluate_acceleration.py --run-id exp3_report_my_run
python evaluation/retrieval/code/render_acceleration_charts.py --run-dir evaluation/
retrieval/results/runs/exp3_inference_acceleration_20261025
python -m unittest discover -s evaluation/retrieval/tests -p test_acceleration.py -v
```

代码 5 复算、重绘与逻辑测试入口

默认拒绝覆盖已有 run，需显式 `--overwrite` 或换新 `--run-id`；所有路径按仓库根目录解析，输出统一到 `evaluation/retrieval/results/runs/`。

3.6 测试优化

测试与优化集中在以下方面：

1. FP16 导出纠错：直接对 `.half()` 权重导出 ONNX 会因 LayerNorm 固定 FP32 而报 mixed dtype，改为先导 FP32 再在图层面转 FP16，并通过固定输入校验嵌入一致性；

2. 版本耦合排查：`onnxruntime-gpu` 1.31 需要 CUDA 13、与本机 CUDA 12.8 不匹配（加载 `libcublasLt.so.13` 失败），锁定 1.22.0；TensorRT 必须为 10.x，因为 ORT 1.22 链接的是 `libnvinfer.so.10`，装 TRT 11 会回退到 CPU；
3. 线程对齐纠偏：早期 PyTorch 仅用 4 线程、ORT 用 8 线程，得到虚高的 1.66 倍；对齐为 8/8 后 CPU 后端实际仅约 1.06 倍，据此否定了“CPU 上有实质加速”的误判；
4. 引擎冷启动与稳态口径：TensorRT 首次构建引擎需 5–20 分钟（不计入稳态延迟），显式启用引擎缓存后后续运行复用，测速取冷启动之后的稳态值；
5. provider 回退防护：会话构建后校验实际提供器，检出回退即判该次结果无效，防止把 CPU 回退结果当作 TensorRT；
6. 基线口径统一：统一使用自研评测的 58.5653 作为共同基线，避免与实验二官方脚本的 58.67 混用；
7. 逻辑测试：8 项单元测试验证复算与判定逻辑，覆盖阈值边界（等于 40%/50%/1.2×/1pp/0.5pp 均判通过）。

四、实验关键技术与难点

FP16 的正确导出是本实验的第一个工程难点。cn_clip 的 LayerNorm 在模型定义中把输入强制转为 FP32，直接对半精度权重导出 ONNX 会触发 mixed dtype 错误。解法是把“精度转换”从 PyTorch 侧移到 ONNX 图层面，用 `convert_float_to_float16(keep_io_types=False)` 统一转换，代价是推理时输入也必须是 FP16。

环境版本耦合是第二个难点。ONNX Runtime 的 GPU 版本与 CUDA 版本强绑定，其中 CUDA 执行提供器要求 `onnxruntime-gpu` 与 CUDA 12.8 匹配（1.22.0），TensorRT 执行提供器又要求 TensorRT 10.x，因为 ORT 1.22 链接的是 `libnvinfer.so.10`。任一版本错配都会静默回退到 CPU，使“加速比”失去意义，故在会话构建后加入提供器校验。

避免“假加速”是第三个难点。CPU 上 PyTorch 与 ONNX Runtime 的线程数必须对齐，否则单方面增加 ORT 线程即可造出 1.66 倍的假象；计算图上还有 Memcpy 节点与部分算子回退 CPU 等因素，也会削弱理论收益。为此规定同硬件、同精度、同线程三条硬约束。

测量口径的一致性是第四个难点。batch 内 32 候选的验证准确率不能与 934 条全候选的 R@K 混比；自研脚本与实验二官方脚本的输出在相同精度下也有约 0.1pp 的口径差（官方 58.67、自研 FP32 58.5653）；加速比的基数必须取各自文件内的同精度 PyTorch 对照，而不能跨设备换算。任何一处混用都会放大或掩盖真实差异。

量化维度的结构性教训同样构成难点。动态量化只替换 `nn.Linear`，实测中 108 个 Linear 仅量化 96 个；文本塔的 72 层 Linear 被整塔改造，而图像塔的注意力 `in_proj`（裸 Parameter）与 `out_proj`（子类 `NonDynamicallyQuantizableLinear`）未被覆盖。结果是“只量化

文本塔”就带来约 −49.57pp 的崩塌，“只量化图像塔”为 −13.06pp。这说明对本双塔模型而言，量化误差主要落在文本塔，也解释了为何本轮改走精度无损的推理加速路线。

五、实验结果与分析

本部分先说清优化对象的规模与精度代价，再给推理加速的速度与精度结果，随后分析“预期目标与实际结果”的差异、数据与模型特性对优化的影响，并补充一组单独统计的历史端到端参考，最后汇总口径与限制。

（一）模型优化对比分析

模型规模与文件体积是“优化对比分析”的基础对象，见表 1 与表 2。

塔	参数量	理论 FLOPs（batch=1）
图像塔（ViT-B-16）	86,192,640	33.70 G
文本塔（RoBERTa）	102,070,272	8.93 G
合计	**188,262,912**	**42.63 G**

表 1 双塔参数量与理论前向计算量。FLOPs 口径为 batch=1、图像 224×224、文本长度 52，仅计 MatMul/Conv，不含 LayerNorm/GELU/Softmax/Embedding 查表

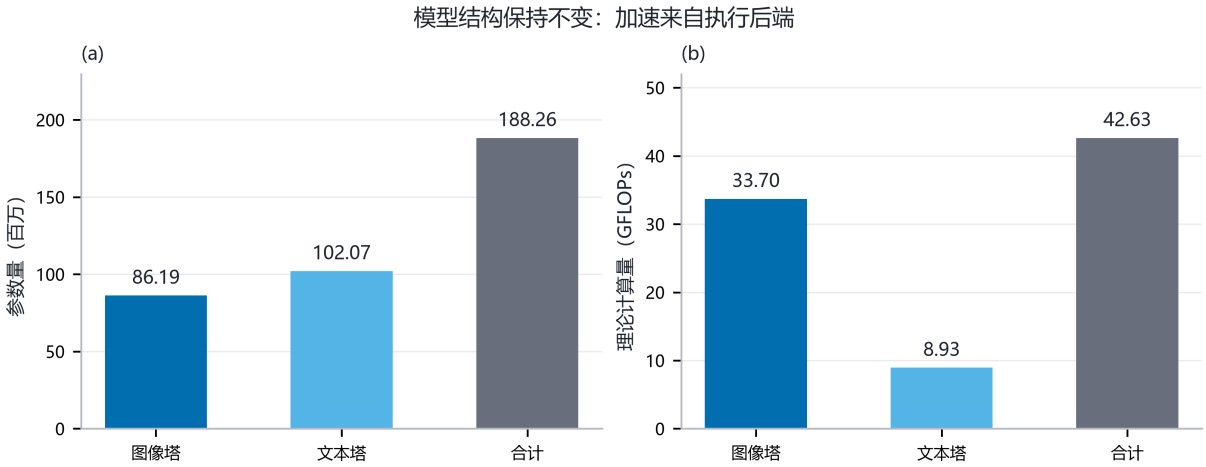


图 2 双塔参数量与理论计算量（图像塔占 FLOPs 约 79%）

对象	精度	体积/MiB	说明
文本塔 ONNX	FP32	389.58	导出报告仅保留最后导出的文本塔，不能当双塔 ONNX 总大小
文本塔 ONNX	FP16	194.97	同上，约为 FP32 的一半
纯权重	FP32	718.29	以纯 FP32 权重为 100% 基准，原始 ckpt 含优化器动量不可作分母
纯权重	FP16	359.21	约为 FP32 的 50.0%

表 2 模型文件体积（MiB = 2^20 字节；文件大小不等于峰值显存，也不等于 TensorRT 引擎缓存大小）

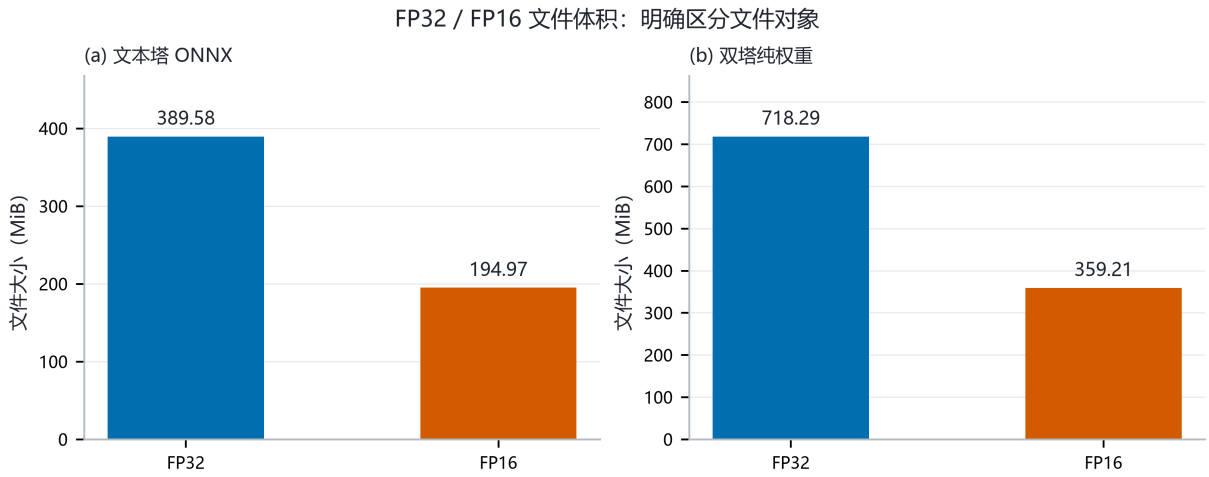


图 3 文本塔 ONNX 与纯权重的 FP32/FP16 文件体积

就“优化对比”而言有两点必须点明。其一，本轮主线是推理加速（后端与计算图优化），不改变模型结构与参数，因此三档精度（FP32/FP16/INT8）的参数量与理论 FLOPs 完全相同——表 1 中“计算量”一行三档同值不是漏填，而是结论本身：量化只改数据位宽与访存，不改运算次数。其二，作为历史探索的 INT8 动态量化把纯权重压到 313.88 MB（相对 FP32 的 43.7%，因只覆盖 `nn.Linear`，未及理论下限 25%），但代价是 R@1 大幅下降（见五（四）），未达任务书“精度损失 ≤2%”的要求，故本轮不采用。

精度损失分析以误差分布热力图呈现（图 4）：由于源凭证不含 TensorRT 分类别原始排名，本轮以“后端 × R@K 指标”的整体差值作图，色标固定 ±1pp，最大绝对差为 0.4283pp；这不是“9 类别 × TensorRT 损失”的热力图（9 类别的 FP32 基线另见图 9）。

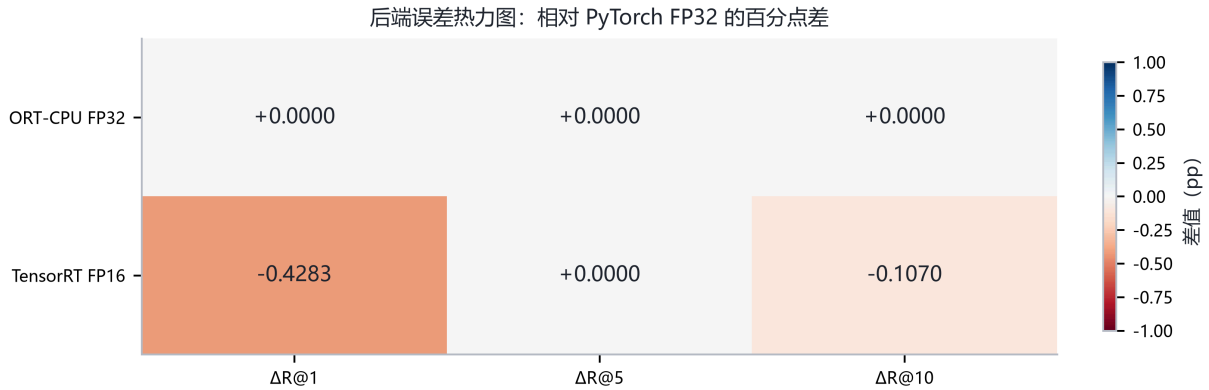


图 4 后端 × R@K 指标的精度误差热力图（色标 ±1pp）。整体最大绝对差 0.4283pp；源凭证无 TRT 分类别结果，故不作分类别损失声明

（二）推理加速效果验证

速度与精度的完整结果见表 3、表 4、表 5，对应图图 5、图 6、图 7 与验收仪表盘图 8。

后端	batch	PyTorch/ms	引擎/ms	加速比	延迟降低	吞吐提升	判定
ORT-CPU	1	90.56	85.24	1.062×	5.87%	+6.36%	未达标
ORT-CPU	32	2532.61	2554.38	0.991×	−0.86%	−0.79%	未达标
ORT-CUDA	1	4.98	4.54	1.097×	8.84%	+9.81%	未达标
ORT-CUDA	32	50.90	54.55	0.933×	−7.17%	−6.70%	未达标
TensorRT	**1**	**4.59**	**1.94**	**2.366×**	**57.73%**	**+136.58%**	达标
TensorRT	**32**	**50.43**	**23.84**	**2.115×**	**52.73%**	**+111.52%**	达标

表 3 图像塔 forward 的同硬件配对速度（中位数，预热 10 次、计时 20 次）。延迟降低为正表示延迟下降；各行为各自文件内的同精度 PyTorch 基线

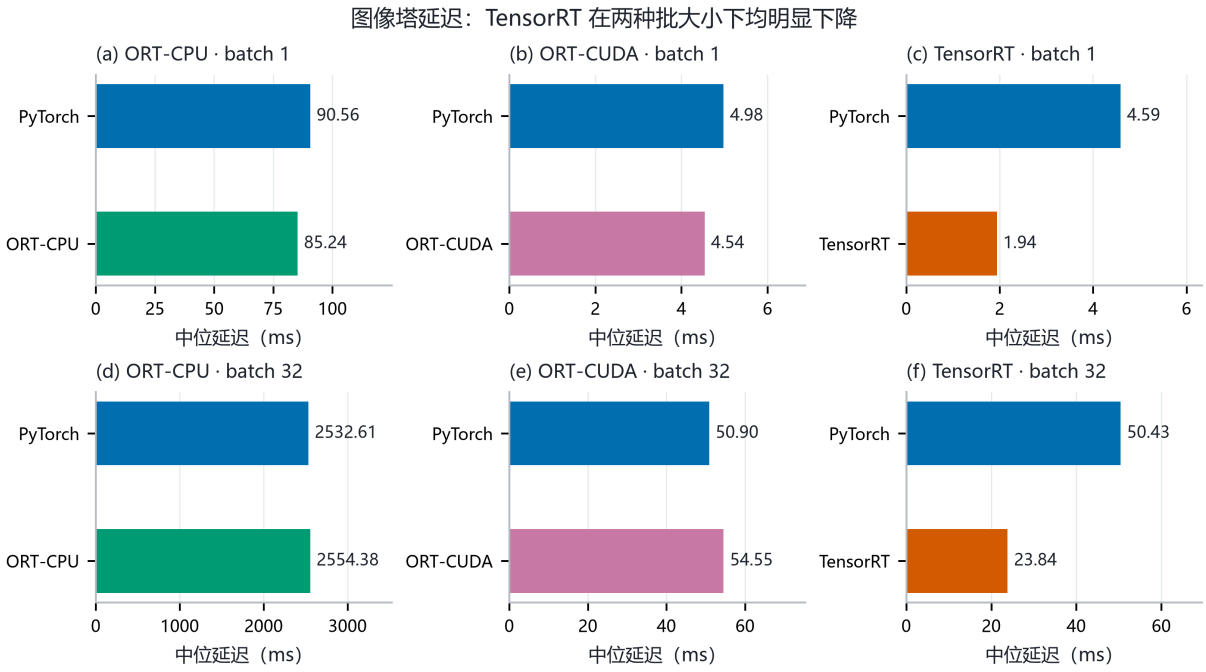


图 5 图像塔延迟对比（batch 1 与 32；各子图独立横轴，不能跨硬件比较）

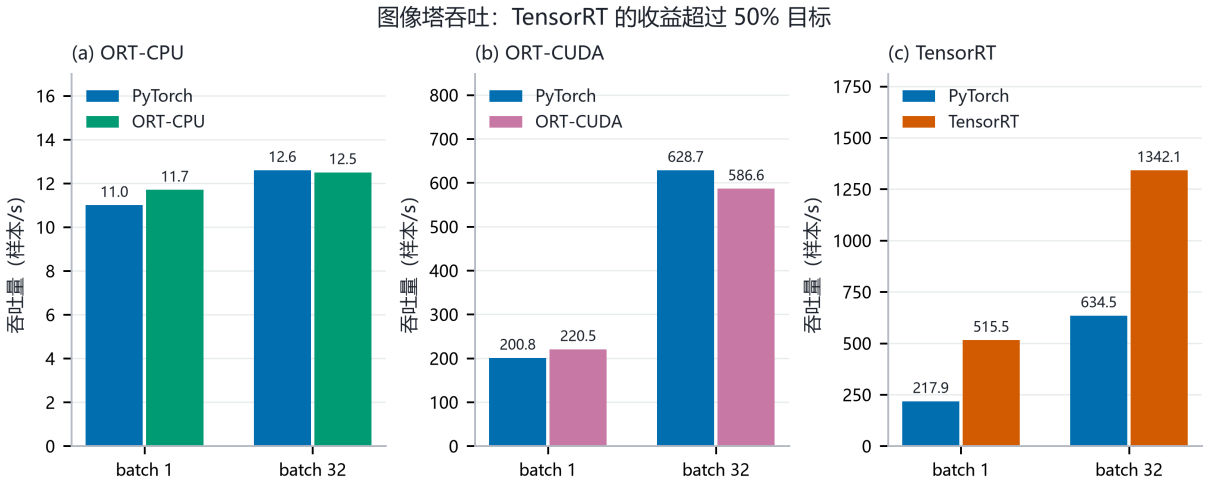


图 6 图像塔吞吐对比（样本/秒）；ORT 后端未达 50% 目标，TensorRT 明显超出

后端	R@1/%	R@5/%	R@10/%	mean_recall/%	ΔR@1/pp
PyTorch FP32（基线）	58.5653	89.2934	96.6809	81.5132	+0.0000
ORT-CPU FP32	58.5653	89.2934	96.6809	81.5132	+0.0000
TensorRT FP16	**58.1370**	**89.2934**	**96.5739**	**81.3348**	**−0.4283**

表 4 双塔文搜图精度一致性（934 条查询、候选池 934）。基线为自研评测的 PyTorch FP32（58.5653），不混用实验二官方脚本的 58.67

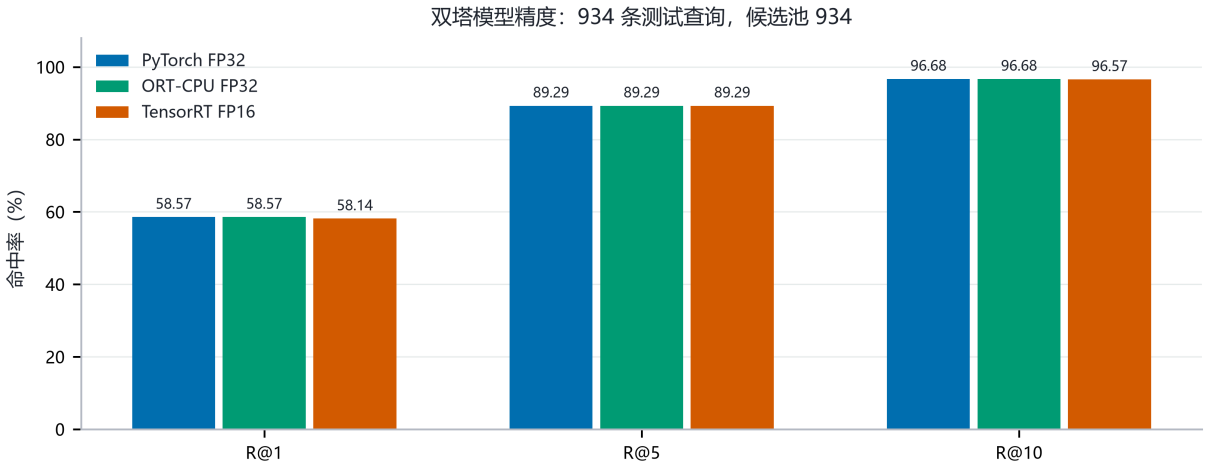


图 7 三后端双塔检索 R@K 对比（R@1 净差 −0.4283pp）

任务	项目	实测	目标	判定
3.1.4	延迟降低 batch=1	57.73%	≥40%	通过
3.1.4	吞吐提升 batch=1	136.58%	≥50%	通过
3.2.1	计算图加速比 batch=1	2.366 倍	≥1.2 倍	通过
3.1.4	延迟降低 batch=32	52.73%	≥40%	通过
3.1.4	吞吐提升 batch=32	111.52%	≥50%	通过
3.2.1	计算图加速比 batch=32	2.115 倍	≥1.2 倍	通过
3.1.4	模型级 R@1 绝对差	0.4283 pp	≤1 pp	通过
3.2.1	模型级 R@1 绝对差	0.4283 pp	≤0.5 pp	通过

表 5 验收目标对照（8 项全部通过）。pp 指百分点；速度项范围为图像塔 forward，精度项为双塔 934 条测试

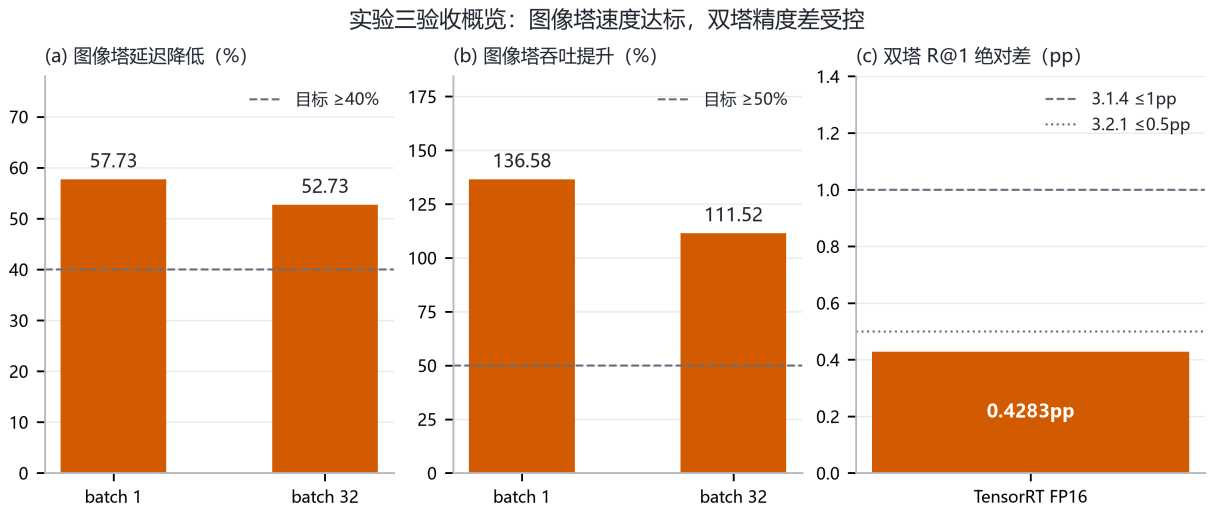


图 8 综合验收仪表盘：延迟降低、吞吐提升与双塔 R@1 绝对差对照阈值

结果解读：TensorRT 后端把图像塔 batch=1 的中位延迟从 4.59 ms 降到 1.94 ms、batch=32 从 50.43 ms 降到 23.84 ms，对应 2.366 倍与 2.115 倍加速，延迟降低 57.73% 与 52.73%，记录吞吐分别提升 136.58% 与 111.52%，两项速度目标均达成。ORT-CPU 与 ORT-CUDA 的提升均在 10% 以内（batch=32 甚至略慢），保留在表中作为不成功的对照。精度方面，ORT-CPU（FP32）与基线完全一致（0.00pp），TensorRT（FP16）R@1 为 58.1370，净差 −0.4283pp（约相当于 934 条中的 4 条 Top-1 净命中差），满足 ≤1pp 的一致性要求。需要强调：速度数据只覆盖图像塔 forward，不能据此宣称完整检索请求取得相同加速。

（三）预期目标与实际结果的差异

按“预期目标 vs 实际结果”逐项对照，可得到有价值的判断：

1. ORT-CPU 未达预期。原以为“CPU 上 ORT 对 Transformer 有算子融合与多线程”是零成本收益，实测对齐线程后仅 1.06 倍（batch=32 为 0.99 倍）。这说明 CPU 上 PyTorch 本已充分优化，纯后端替换难有实质收益；若确需 CPU 部署提速，应转向 OpenVINO[12] 等针对 CPU 的图优化路线（本轮未实测）；
2. ORT-CUDA 未达预期。GPU 上的 CUDA 执行提供器在 batch=32 反而慢了 7.17%，与计算图中存在 Memcpy 节点、部分算子回退 CPU 有关；仅靠“换到 GPU 跑”不足以提升，需要引擎级的 kernel 重写与层融合；
3. TensorRT 达到预期并超出速度目标。层融合与 FP16 使其取得 2.1–2.4 倍加速，是全轮唯一同时满足延迟与吞吐目标的后端；
4. 精度目标稳定达成。FP32 引擎与基线的 0.00pp 说明“换引擎不改变数学结果”，FP16 的 −0.4283pp 是后端与精度配置共同改变后的净差，因缺少逐算子/逐查询对照，不能断言其全部来自 FP16；
5. 抖动提示：同配置重复测量时加速比在 2.1–2.5 倍间波动（约 ±5%），batch=1 单次仅约 2 ms 使抖动更明显，本轮取一次稳态运行。

与实际应用的关联：对本课题的检索服务而言，真正可用的加速来自 TensorRT 路径，且收益集中在图像塔——这与图像塔占双塔 FLOPs 约 79% 的事实一致。因此部署建议是：在有 GPU 的环境用 TensorRT-FP16 承载图像塔编码，文本塔保持 ONNX Runtime；在纯 CPU 环境则应评估 OpenVINO 而非依赖 ORT 后端替换。

（四）关联分析：数据特点与模型结构对优化的影响

任务书要求分析实验一数据特点对量化误差的影响、以及实验二模型特性对剪枝敏感性的影响。前者可从已冻结的量化探索中得到数据支撑，后者本轮未做剪枝，只能给方向性推断。

实验一数据特点对量化误差的影响见表 6（基线用 9 类别的 FP32 结果，误差用已冻结的 INT8 逐层动态量化结果作对照）。

类别	样本数	FP32 R@1	INT8 R@1	ΔR@1/pp
包	123	57.72	14.63	−43.09
水杯	195	60.51	21.03	−39.49
家居用品	23	91.30	52.17	−39.13
运动服	66	71.21	34.85	−36.36
厨房用品	28	71.43	39.29	−32.14
女装	32	93.75	62.50	−31.25
鞋	336	51.79	21.13	−30.65
男装	102	51.96	27.45	−24.51
手机壳	29	44.83	34.48	−10.34

表 6 九类别 FP32 基线与冻结量化误差（候选池始终为完整 934；INT8 为逐层动态量化的历史探索，仅作误差关联说明，非主线结果）

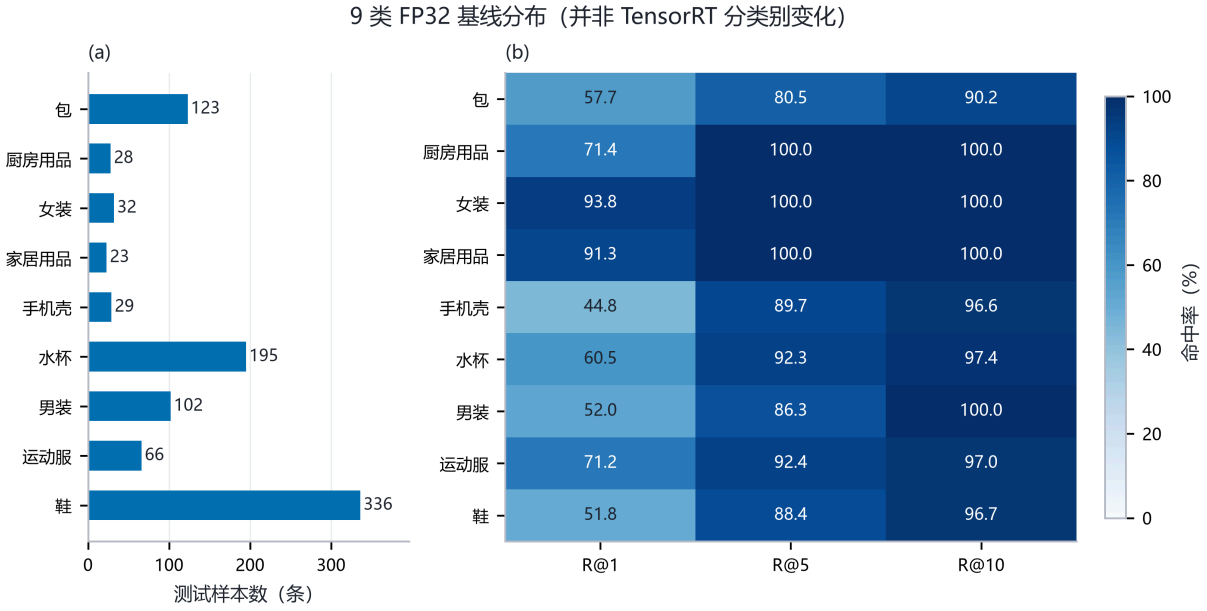


图 9 九类别样本规模与 FP32 基线分布（候选池不缩小；此图不是 TensorRT 分类别变化）

从表 6 可读出三点：量化误差在类别间并不均匀，跌幅从手机壳的 −10.34pp 到包的 −43.09pp，相差约 4 倍；量化误差较大的类别（包、水杯、家居、女装）多依赖颜色、材质、细分类等细粒度属性，而对这些细节的区分恰被低位宽抹平，故“越依赖细粒度属性、越易受量化影响”；鞋占 934 条的 36%，其 −30.65pp 的跌幅对全局拖累明显，是全局跌幅的主要来源之一。这组数据说明：在同一数据集内，量化误差的分布由类别的属性结构决定，而不是与基线难度单调相关（手机壳基线最低却最抗量化）。

模型特性对剪枝敏感性的影响本轮未做剪枝实验，不能给出剪枝敏感性的实测结论。仅能从量化探索推断方向：文本塔（RoBERTa，72 层 Linear）被量化整塔破坏时 R@1 下跌约 49.57pp，而图像塔（ViT，注意力投影未被动态量化覆盖）仅为 −13.06pp，说明两塔对参数扰动的敏感性不同；ViT 的 `in_proj`（裸 Parameter）与 `out_proj`（`NonDynamicallyQuantizableLinear`）结构不利于按模块类型整体替换，预示着后续若做结构化剪枝或蒸馏，文本塔可能是更敏感、也更需要保护的一侧。以上均为间接依据，剪枝结论须以实测为准。

（五）历史端到端查询参考（单独统计）

为检查实验二微调权重接入既有索引后的检索行为，仓库保存了一轮旧索引的 AI 试点检索（132 条有效查询、1325 条 AI 相关性标签）。本轮从历史 Top-10 排序复算指标，结果见表 7 与图 10。

查询模式	查询数	Precision@10	Recall@10	nDCG@10	MRR@10
全部	132	0.3189	0.9784	0.8682	0.7741
文字	44	0.3000	0.9407	0.8683	0.8646
图片	45	0.3911	1.0000	0.8740	0.8035
图文	43	0.2628	1.0000	0.8621	0.6508

表 7 历史 AI 试点检索的分模式指标（旧索引历史排序复算，未用本轮 ONNX/TensorRT 重新检索）

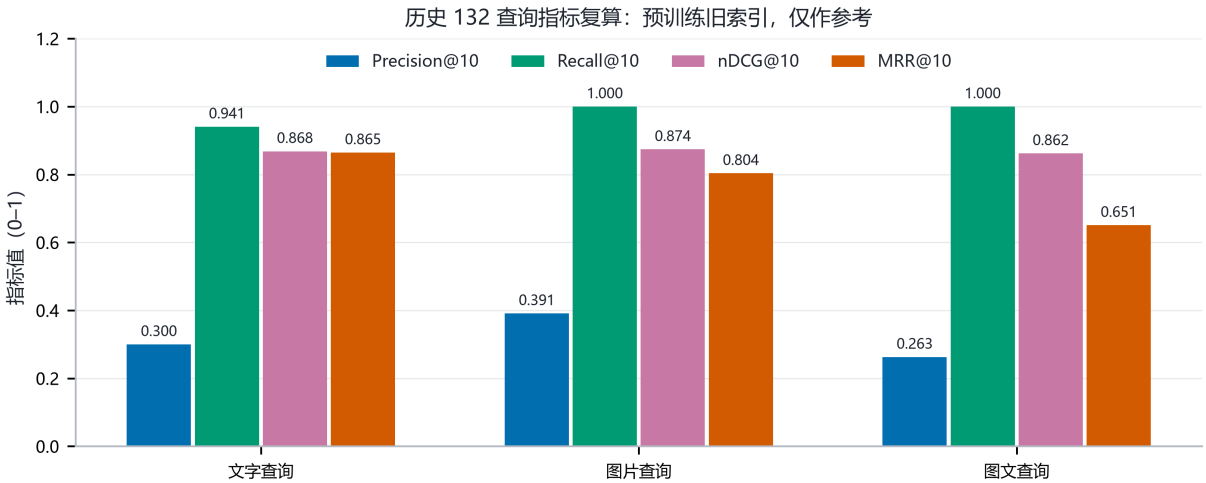


图 10 历史查询分模式检索指标（仅作历史端到端参考，不代表加速后的结果）

这组资料属于预训练索引的 AI 试点验证集，与 934 条精确配对测试是两套口径，不能混算。相关度以 ≥2 为相关，未标注商品按现有脚本当作不相关；AI 试点标签由模型生成与审核，不是独立人工真值。因此这组结果只能用于说明既有系统的端到端基线，不能用来证明本轮 TensorRT 的端到端收益。

（六）口径、限制与未完成项

为使结论可被严格复核，本轮如实标注限制：

1. 速度只覆盖图像塔 forward，不含预处理、FAISS、商品过滤、重排、网络与界面，不等于整系统或用户请求的端到端延迟；
2. 精度一致性数据是整体 R@K，缺少 TensorRT 分类别与逐样本排名，不能生成分类别损失结论，也不能指出具体有多少条查询发生变化；
3. 单轮预热 10 次、计时 20 次，无独立重复实验的置信区间；表格中的 P90 仅描述同轮尾延迟；
4. 历史 132 查询只复算原排序，未用微调权重或 TensorRT 重新检索，不用于 TensorRT 达标判断；
5. 源测速凭证未独立记录 GPU 型号，本轮不补写；
6. `export_report` 仅保留最后导出的文本塔，双塔 ONNX 总体积不能由此推出；
7. 量化、4-bit、剪枝、蒸馏、资源占用与鲁棒性均不在本轮主线；现场演示/录屏尚需本人在训练机完成；正式 Typst/PDF 由报告成员整合。

六、实验总结与展望

本实验以实验二微调的 Chinese-CLIP 双塔模型为对象，完整走通了“ONNX 转换 → 计算图与后端优化 → 同硬件配对测速 → 精度一致性验证”的算法优化链路。在图像塔 forward 范围、双塔 934 条整体 R@1 一致性的明示口径内，任务 3.1.4 的速度与精度条件全部达成，任务 3.2.1 按本轮口径（加速比 ≥1.2、R@1 差 ≤0.5pp）亦达成：TensorRT-FP16 把图像塔 batch=1/32 中位延迟分别降至 1.94/23.84 ms（加速 2.366×/2.115×，延迟降低 57.73%/52.73%），吞吐提升 136.58%/111.52%；ORT-CPU 与 ORT-CUDA 均未达标，作为对照保留；TensorRT-FP16 的双塔 R@1 为 58.1370%，相对同源 FP32 基线净差 −0.4283pp，满足 ≤1pp。

通过本次实验，进一步理解了“性能-精度-资源”的权衡：量化维度虽完整实测却因误差过大而被冻结（最好一档仍下跌 13.06pp），说明并非所有轻量化手段都适配双塔对比学习模型；而推理加速在不改结构、不动参数的前提下取得 2 倍以上加速，并把精度差控制在 0.4pp 量级，是更务实的选择。也认识到“换后端即可加速”是常见误区——CPU 上 ORT 收益有限、GPU 上 ORT-CUDA 甚至略慢，真正的加速来自引擎级的层融合与 kernel 重写。

后续工作将从四个方面推进：

1. 补齐文本塔独立测速与端到端延迟，把“图像塔 forward 加速”扩展为可验证的整系统收益；
2. 在训练机补导出 TensorRT 分类别结果，得到 9 类别 × TensorRT 的误差分布，替代当前的“后端 × 指标”近似；
3. 在现有 ONNX 图上尝试 INT8 静态量化（QDQ + 校准）与量化后微调，重新评估“精度损失 ≤2%”是否可达，并与本轮 FP16 结果对照；
4. 面向 Agent 的工具化（实验四、五），把 TensorRT 编码与检索封装为可调用工具，并对接前后端与大模型编排。

七、参考文献

[1] JACOB B, KLIGYS S, CHEN B, 等. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713. DOI:10.1109/CVPR.2018.00286.

[2] GHOLAMI A, KIM S, DONG Z, 等. A Survey of Quantization Methods for Efficient Neural Network Inference[J/OL]. arXiv preprint arXiv:2103.13630, 2021. https://arxiv.org/abs/2103.13630.

[3] HINTON G, VINYALS O, DEAN J. Distilling the Knowledge in a Neural Network[EB/OL]. (2015). https://arxiv.org/abs/1503.02531.

[4] ONNX OPEN SOURCE COMMUNITY. Open Neural Network Exchange (ONNX)[EB/OL]. (2026). https://onnx.ai/.

[5] MICROSOFT CORPORATION. ONNX Runtime: Performance Tuning[EB/OL]. (2026). https://onnxruntime.ai/docs/performance/.

[6] NVIDIA CORPORATION. NVIDIA TensorRT: Best Practices[EB/OL]. (2026). https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html.

[7] MICIKEVICIUS P, NARANG S, ALBEN J, 等. Mixed Precision Training[C/OL]//International Conference on Learning Representations (ICLR). 2018. https://openreview.net/forum?id=r1gs9JgRZ.

[8] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, 等. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale[C/OL]//International Conference on Learning Representations (ICLR). 2021. https://openreview.net/forum?id=YicbFdNTTy.

[9] CUI Y, CHE W, LIU T, 等. Pre-Training with Whole Word Masking for Chinese BERT[J/OL]. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2021, 29: 3504-3514. DOI:10.1109/TASLP.2021.3124365.

[10] YANG A, PAN J, LIN J, 等. Chinese CLIP: Contrastive Vision-Language Pretraining in Chinese[EB/OL]. (2022). https://arxiv.org/abs/2211.01335.

[11] JOHNSON J, DOUZE M, JÉGOU H. Billion-scale Similarity Search with GPUs[J/OL]. IEEE Transactions on Big Data, 2021, 7(3): 535-547. https://arxiv.org/abs/1702.08734. DOI:10.1109/tbdata.2019.2921572.

[12] INTEL CORPORATION. OpenVINO Model Optimization Guide[EB/OL]. (2026). https://docs.openvino.ai/.

附录 A 实验代码与结果目录

本附录列出本实验实际使用或用于复核的代码与结果文件位置。大体积权重、ONNX 与 TensorRT 引擎缓存按仓库约定不入 Git；报告保留其路径、字节数与可重建命令，源文件以仓库当前版本为准。

A.1 关键代码入口

路径	作用	报告对应内容
`scripts/optimize/export_onnx.py`	单塔导出 ONNX（图像塔固定 224×224、文本塔动态序列），先 FP32 再图层面转 FP16，并校验嵌入余弦一致性。	3.5.1、四
`scripts/optimize/bench_runtime.py`	图像塔延迟/吞吐基准，同硬件配对、线程对齐、warmup/iters 可配，含 provider 回退检测与 TensorRT 引擎缓存。	3.5.2、3.5.3、五（二）
`scripts/optimize/eval_onnx.py`	两 ONNX 塔在 934 条 test 上算 R@K 并与同源 FP32 基线比对，输出一致性凭证。	3.5.3、五（二）
`scripts/optimize/bench_flops.py`	统计两塔参数量与理论 FLOPs（仅 MatMul/Conv）。	3.5.3、五（一）
`scripts/optimize/eval_by_category.py`	在完整 934 候选池上按 9 类拆分 R@K（分类别基线数据源）。	五（一）、五（四）
`scripts/optimize/quantize.py`	落盘 FP32/FP16/INT8 三份权重并量体积（冻结的量化探索）。	五（一）、五（四）
`scripts/optimize/eval_quant.py`	任一精度在 934 条 test 上重算 R@K（冻结的量化探索）。	五（四）
`evaluation/retrieval/code/evaluate_acceleration.py`	读取训练机凭证复算加速比、精度差与阈值判定，写出表格、汇总与来源清单。	3.5.4、五（二）
`evaluation/retrieval/code/render_acceleration_charts.py`	从复算表格独立重绘 9 张中文图（PNG/PDF/SVG/figure.json）。	五、附录 A.2
`evaluation/retrieval/tests/test_acceleration.py`	8 项复算与判定逻辑测试（配对、回退、跨设备、线程、阈值边界等）。	3.5.4、3.6
`evaluation/retrieval/scripts/run_exp3_evaluation.ps1`	Windows 复算入口，不绑定个人 Python 路径。	3.5.4
`evaluation/retrieval/configs/exp3_acceleration.json`	本轮来源选择、后端分组、阈值与历史排除项。	3.5.4、五（六）

表 8 实验三关键代码入口与报告对应关系

A.2 结果与凭证文件清单

统一交付目录为 `evaluation/retrieval/results/runs/exp3_inference_acceleration_20261025/`，逐文件用途见其中的 `测试结果说明.md`，文件清单见表 9。

文件/目录	说明
inputs/	16 份输入快照与 source_manifest.json（原路径、快照路径、原始字节 SHA-256 与统一 UTF-8/LF 文本 SHA-256）
tables/runtime_comparison.csv	每组 batch 的配对速度、吞吐、加速比与目标判定（重绘与写表的主要数据）
tables/accuracy_comparison.csv	双塔 R@K 与相对同源基线的百分点差
tables/acceptance_checks.csv	3.1.4 / 3.2.1 的阈值、实测值与通过状态（8 项）
tables/model_scale.csv、model_sizes.csv	参数量/FLOPs 与不同对象的 FP32/FP16 文件大小
tables/category_baseline.csv	9 类的样本数、FP32 基线 R@K
figures/01–09（PNG/PDF/SVG/figure.json）	9 张中文图及其结论、口径、配色与审计记录；图表说明.md 逐张说明用法
reference_retrieval/	132 条历史查询的逐查询与分模式指标复算（与主线分开）
summary.json	机器可读结论、各任务判定与 7 条限制
run_metadata.json	生成时间、源提交、执行范围、未执行项与排除来源
docs/optimize/runs/2026-10-24_exp3-speed/	训练机原始测速、导出、精度与 FLOPs 凭证（保留原件，不改动）
docs/optimize/runs/2026-10-23_exp3-quant/	冻结的量化探索凭证（体积与精度，仅作历史说明）

表 9 实验三结果与凭证文件清单

A.3 重要约束与复现注意事项

复现分两条链路。模型侧（需训练机、权重与 ONNX/TensorRT 环境）：依次执行 export_onnx.py（双塔、FP32/FP16）、bench_runtime.py（三种后端、batch 1/32）、eval_onnx.py 与 bench_flops.py，产物为 JSON 凭证。报告侧（仅需 numpy/matplotlib/Pillow）：执行 evaluate_acceleration.py 从凭证复算，再用 render_acceleration_charts.py 重绘。运行前须挂 LD_LIBRARY_PATH 指向 TensorRT 与 CUDA 库；Conda 环境须正常激活，避免底层数值库 DLL 路径缺失；Linux 绘图需 Noto Sans CJK SC 等中文字体。

复现时需注意若干约束。epoch3.pt 约 2.2 GB、双塔 ONNX 与 TensorRT 引擎缓存均由 .gitignore 排除，换机器需按 docs/optimize/README.md 重新导出，且 TensorRT 首次构建引擎需 5–20 分钟。速度对比必须同硬件、同精度、同线程，不可跨 CPU/GPU 换算加速比；精度必须使用与基线相同的 934 条测试与 934 候选池。交付文档中的数字一律以本目录 tables/ 与 summary.json 为准，不应手工改写。现场演示/录屏需持有训练机与模型文件的人执行，本轮已提供操作步骤但尚未录制。